EXP- 11
DATE 20/8/26

# EXPERIMENT ON PACKET CAPTURE TOOL WIRESHARK

Aim

Capturing an analysis packets using wireshark tools

1) Aim

Create a filter to display only TCP/UDP packets, inspects the packets and provide the flow graph

PROCEDURE:-

* Select LAN connection in wireshark
* Go to Capture → option
* Select stop capture automatically after 100 packets
* save the packets

2 Aim:-

Create in filter to display only ARP packets and inspect the packets

PROCEDURE:-

* Go to capture
* Set Stop Capture automatically after 100 packets
* Then click start Capture
* Save the packets

3) Aim:-

Create a filter to display only DNS packets & provide the flow graph.

PROCEDURE:-

- Go to capture → option
- Select stop capture automatically after 100 packets
- Then click start capture
- Search DNS packets
- Save the packets

AIM:-

Create a filter to display only HTTP packet & repeat the packets

PROCEDORE:-

- Select LAN connection in wireshark
- Go to capture → option
- Then click start capture.
- Select HTTP packets in search
- Save the packets.

AIM:-

To create a filter to display only IP/ICMP packet and inspect the packet

PROCEDURE:-

- Select the LAN Connection in wireshark
- Go to Capture → option
- click starts capture.
- Search for ICMP/IP packets
- Save the packets.

Experiment 12

Date:- 17/9/26

# Discovery of Live Host using ARP, ICMP, TCP, UDP.

AIM:-

To discover the live host in a network using Nmap scans such as ARP, ICMP, TCP and UDP on the TryHackMe

OBJECTIVE:-

To understand how nmap identify active system before performing further network scanning.

SOFTWARE:-

Nmap.
TryHackMe Nmap loom.
Computer with network success

THEORY:-

Nmap is a free and open source network scanning tool used to discover live hosts and identify network services it was created

Nmap uses different live hosts using different protocols

1) ARP scan - uses ART scans request to find hosts on same subnet

2) ICMP scan - use icmp request and replies to identify active host.

3) TCP scan - uses TCP request and packets to specified ports & check the respons.

4) UDP scan — send UDP packets & can identify an active host through an ICMP port

OBSERVATION:

→ ARP request can discover hosts within the same subject.

→ ARP cannot cross routers because it operates at the link layer.

→ An ARP ping generally requires a Previledged account.

→ UDP ping can identify an active host when an ICMP port unreachable is received

→ The -sn option allows Nmap to perform host discovery without conducting a port scan..

Exp: no: 13 Sliding window protocol

Date: 27/8/26

Aim:-

To write a programe no implement flow control at the data link layer using the Sliding window protocol and simulate the flow of frames from one node no another.

SENDER CODE:-

```
message = input ("enters the message")
window size = int (input("enter window size")
frames = [ ]
for i, ch in enumerate (message):
frames . appeard ((i+1, ch,))
print ("in frames created")
for no data in frames
print (f formal {no} = {data})
with open (sender - Buffet, text", "w") as
files for no, data in frames:
file write (f"{no} {data} in")
base = 0
while base < len (frames) =
print ("in current window")
for i in range (base, min (base + window-size)
len (frames)))
print (f "[{ frames (i)(0).
{ frames (i.) (i) } }, end = " }
```

```
try:
    with open("receiver_Buffer.txt", "r")
response = file.readline().strip()
except FileNotFoundError
  response =
response.startswith("ACK")
ack = int(response.split()[1])
print(f"ACK {ack} received")
base = ack
Open("receiver_Buffer.text", "w").close()
if response starts with("NACK")
nack = int(response.split()[1])
print(f"NACK {nack} received")
(f"retransmitting frame {nack}...")
print("All frames sent")
input("Run receiver program & press ENTER TO continues)
print("All frames transmitted)
RECEIVER CODE:-
expected_frame = 1
while true:
try
  with open("sender_Buffer.text", r)
    as file,
frames = file.readlines()
except FileNotFoundError:-
Print("sender_Buffer.txt not found")
```

```
if not frames
    print("no frames available") break
print("in frames received from sender")
for line in frames:-
    parts = line.split()
    if len(parts) != 2
        continue
frame_no = int(parts[0])
data = parts[1]
print(f"Frame {frame_no}: {data}")
if frame_no == expected_{frame_no}
    received"}
with open("receiver_Buffer.txt",'w') as
    act file:
act_file.write(f"ACK{frame_no}")
Print(f"ACK{frame_no} sent")

    print("wrong frame received")
with open(receiver_Buffer.txt'w)
act file.write(f"NACK{expected_
                        frame} sent.
break
if expected_frame == len(frames):
print("All frames received") break
input("press ENTER to check next
        frame")
```

Input :-

Enter the message : Hello
Enter window size : 3

Output :-

frames : created :-

frame 1 : H
frame 2 = E
frame 3 = L
frame 4 = L
fram 5 = O

frames stored in sender-buffer, text current window.

[1 : H] [2 : E] [3 : L]

frames sent
Run receiver program and press ENTER to continue.

Received Output :-

Frames received from sender,
fram 1 = H
Frame 1 receive correctly
Ack 1 sent

Result :-

The sliding wind protocol was successfully implemented using Python and the data transmissions using and acknowledgement process

Exp: 14

# Implementing packet sniffing using raw sockets

Aim:

To implement packet sniffing using raw sockets in Python

ALGORITHM:-

1) Create a RAW socket
2) Bind the socket to the network interface
3) Receive packets continuously
4) Read the packet header
5) Display the source & destination IP addr
6) Display the protocol used
7) Continue until the user stops the program

INPUT:-

Network packets received through the network interface.

CODE:-

```
important
import socket
S = socket.socket(socket.AF_INET, socket.
sock_raw, socket, IPPROTO_IP)
S.bind(("0.0.0.0", 0))
S.setsockopt(socket, IPPROTO_IP, socket
print("packet sniffer started....")
while True:-
packet, addr = S.recvfrom(65565)
ip header =
```

```
src_ip = socket.inet_ntoa(iph[8])
dest_ip = socket.inet_ntoa(iph[9])
print("Source IP:", src_ip)
print("Destination IP:", dest_ip)
print("Packet length:" len(packet))
```

Output:-

```
Packet Sniffer Started.....
Source IP: 192.168.1.10
Destination IP: 192.168.1.1
Packet length: 79
```

Result:-

Thus a raw socket program was successfully implemented to capture & display network packet:

Exp: 15
Date

Server logs

Aim:

To analysis different types of server logs using the weblize

ALGORITHM:-

1) install and open weblizer
2) select the web server log file
3) configure weblizer to read the log file
4) Run weblizer to process the logs.
5) open the generated HTML report
6) Analyse visitors, request, pages, data transfer and HTTP status codes.

Input:-

The web server log file such as access logs.

Output:-

Weblizer generates reports showing

- number of visitors,
- number of reques
- most visited page
- Date transferred
- HTTP status codes

Result:-

Successfully the web log were analysed using weblizer